

TRILHA C — Atendimento

Relatório de entrega · Cards 6.1 + 2.6, 6.2 e 6.3 · JurisAI (build-your-dreams-153)

Data: 09/07/2026 · Autor: Claude (Claude Code) · Branch: `claude/trilha-c-audio-atendimento`

PR #82 MERGED

CI VERDE (ci + edge)

2 migrations aditivas em produção

deploy do edge = passo manual

Os três cards da TRILHA C foram implementados como **três ciclos independentes** (spec → plano → execução dirigida por subagentes com revisão a cada tarefa e uma revisão final da branch inteira), consolidados no **PR #82** (merge commit `76ab819`). Este relatório descreve o que foi entregue, as decisões, o processo de qualidade, e os **passos manuais restantes** para ativar cada feature em produção.

3 CICLOS ENTREGUES	~28 COMMITTS (SPEC+PLANO+CÓDIGO)	2 MIGRATIONS EM PROD (17 DOC-TYPES)
13+ ACHADOS DE REVIEW CORRIGIDOS	1 EDGE FUNCTION NOVA (ISOLADA)	0 MUDANÇAS DE COMPORTAMENTO ATIVAS (TUDO GATED/INERTE)

1. Ciclo 1 — Gravação de áudio do atendimento (6.1 + 2.6)

- O que faz:** grava o áudio real de um atendimento como *prova*, a partir da ficha do cliente (aba “Áudios/Transcrições”), com **segmentação automática em blocos de ~10 min sem interromper** a gravação e **upload incremental** de cada bloco.

Como funciona: um `MediaRecorder` grava sobre um único `MediaStream`; a cada ~10 min há uma rotação interna (stop/restart) que fecha um bloco WebM completo e tocável e começa outro — o usuário nunca reinicia nada. Cada bloco sobe assim que fecha (resiliente a fechar a aba), com fila sequencial e retry por bloco. Buffer/índice/sessão são capturados no closure de cada gravador → fronteiras de bloco corretas independentemente do *timeslice*.

Armazenamento: reusa 100% o padrão de `clientDocuments.ts` — bucket `client-documents` + linha em `client_documents` com `document_type='audio_atendimento'`, agrupadas por `session_id` no `file_path`. Aparece no Histórico do cliente automaticamente.

Arquivos: `src/lib/attendanceAudio.ts`, `src/hooks/useAttendanceRecorder.ts`, `src/components/clients/tabs/chatTabs.tsx` (aba), + `notes` em `clientDocuments.ts`.

Gancho de transcrição: pronto (seam) mas não implementado — é outra track.

2. Ciclo 2 — Resumo do atendimento via LLM (6.2)

- O que faz:** gera um **resumo estruturado via LLM** (problemas, bancos, contratos, empréstimos, tarifas, ações possíveis, documentos solicitados, pendências, orientações, próximos passos) e **salva no cliente**, visível na ficha (aba Resumo) e no Histórico.

Como funciona: uma **edge function nova e isolada** (`attendance-summary`) — deliberadamente separada do `chat-orchestrator` para não arriscar o chat de produção — reúne o conteúdo textual do atendimento, chama o LLM (`gpt-4o-mini` , `temperature 0` , JSON mode) com uma chave resolvida **server-side** (nunca no cliente), normaliza e grava em `client_documents` (`document_type='resumo_atendimento'`).

Anti-alucinação: o prompt instrui “não invente; se não estiver no texto, use ‘não informado’”, e a normalização garante todos os campos presentes com esse default. Falha de parse do LLM é marcada como `fonte:"erro_parse"` (não se disfarça de resumo vazio).

Limitação honesta (documentada)

A transcrição de áudio ainda não existe (é outra track). Logo, hoje a **entrada do resumo é o conteúdo de chat** da sessão do cliente. Um atendimento só-áudio (sem conversa de chat) gera um resumo com todos os campos “não informado” (`fonte:"sem_conteudo"`). Quando a transcrição existir, ela alimenta o mesmo pipeline.

Arquivos: `supabase/functions/attendance-summary/{index.ts, attendanceSummary.ts, attendanceSummary.test.ts}` , `src/lib/attendanceSummaryClient.ts` , `src/components/clients/tabs/infoTabs.tsx` (ResumoTab).

3. Ciclo 3 — Checklist de documentos pelo chat (6.3)

O que faz: um comando no chat como “Para Crefisa, solicite extrato e contrato; registre como pendentes” cria as **pendências documentais** corretas, usando o **mesmo cartão de confirmação editável do card 4.1** (propor → revisar → confirmar → gravar), atrás de uma **flag dedicada**.

Como funciona: uma nova *write-tool* `solicitar_checklist_documental` no `chat-orchestrator` mapeia cada documento (texto livre) para um tipo de pendência (dicionário determinístico e testado) e chama `criar_pendencia` uma vez por documento, com o réu no título. A pendência criada (um `user_task` atribuído, com prazo) é a tarefa de cobrança vinculada ao cliente. O `ActionCard` do 4.1 é reusado sem qualquer mudança de front-end.

Bloqueio no ponto certo: a entrada não trava (gera pendência); o gate/protocolo existente (`auto_liberar_gate_documental`) continua governando o bloqueio — nenhum código de bloqueio novo foi adicionado.

Descoberta que ajustou o escopo (MVP)

A função de banco `client_required_set` **não recebe tipo de ação/réu** — só distingue cooperado/não-cooperado. Um “checklist por réu/ação” via *required-set* não existe no banco e seria schema novo. O MVP entregue usa o motor de pendências existente (defensável e sem schema novo); o *required-set* por réu fica como evolução futura.

Arquivos: `supabase/functions/chat-orchestrator/tools/{docChecklist.ts, docChecklist.test.ts, registry.ts, handlers.ts}`, `chat-orchestrator/index.ts` (flag + gating + humanSummary). **Backend-only** (sem mudança de FE).

4. Migrations aplicadas em produção (aditivas, verificadas)

Todas aditivas, aplicadas via `apply_migration` (não `db push`), recriando o CHECK a partir do conjunto **vivo** de produção + o valor novo — para não regredir o superset (desync repo↔banco conhecido). Verificadas por `pg_get_constraintdef`.

VERSION	MIGRATION	EFEITO	VERIFICAÇÃO
20260709173124	audio_atendimento_doc_type	+ <code>audio_atendimento</code> no CHECK	16 valores, nada perdido
20260709184145	resumo_atendimento_doc_type	+ <code>resumo_atendimento</code> no CHECK	17 valores, audio preservado

Ciclo 3 não exigiu schema novo (reusa `criar_pendencia`). CHECK final de `client_documents.document_type` em produção: **17 valores**.

5. Processo & qualidade

Cada ciclo passou por: *brainstorming* → spec → plano → execução por subagentes (um implementador por tarefa + revisão de spec/qualidade a cada tarefa) → **revisão final da branch inteira** (modelo mais capaz).

Como não há Node/Deno na máquina local, testes/typecheck/build validam no **CI (Vercel + GitHub Actions job edge com deno check / deno test)**.

As revisões pegaram e corrigiram defeitos reais antes do merge — amostra:

ONDE	ACHADO	CORREÇÃO
6.2 edge (Critical)	insert sem <code>uploaded_by</code> (NOT NULL + RLS) → todo insert daria 500	resolve <code>uid</code> via <code>getUser()</code> e grava <code>uploaded_by</code>
6.1 motor	buffer compartilhado entre gravadores dependia do timeslice (fronteiras frágeis)	buffer por-gravador no closure; rotação falha "alto"
6.1 UI	lista "Atendimentos gravados" não recarregava após gravar	recarrega quando todos os blocos concluem
6.1 fila	upload preso se <code>uploadFn</code> lançasse; retry travava	try/catch → estado erro; <code>upsert:true</code> idempotente
Branch (Important)	<code>notes</code> (JSON de máquina) vazava cru na aba Documentos de todos	oculta <code>notes</code> para os tipos de máquina
Branch (Important)	rótulos dos tipos novos faltando (slugs crus na tela)	+ labels "Áudio do atendimento"/"Resumo do atendimento"
CI edge	tipagem do <code>resolveKey</code> quebrava <code>deno check</code> ; <code>includes()</code> em union	tipar como <code>SupabaseClient</code> ; cast <code>readonly string[]</code>

6. Passos manuais restantes (ativação em produção)

Deploy do edge é manual (por decisão do projeto)

O código está mergeado e verde no CI, mas o **deploy das edge functions não foi feito automaticamente** — a briefing designa o deploy do edge como manual, e o `chat-orchestrator` serve o chat ao vivo. Ambas as features estão **inertes** até os passos abaixo.

Para ativar o 6.2 (resumo):

- `supabase functions deploy attendance-summary --project-ref tsltxvswzdnlmvljpryh`
- (a flag `ATTENDANCE_SUMMARY_ENABLED` já é *default ON*; opcionalmente defina `=false` para desligar)

Para ativar o 6.3 (checklist no chat):

- `supabase functions deploy chat-orchestrator --project-ref tsltxvswzdnlmvljpryh` (inerte: a flag está OFF)
- definir a env `CHAT_DOC_CHECKLIST_ENABLED=true` no edge
- adicionar `solicitar_checklist_documental` ao `agents.allowed_tools` do agente da recepção
- (uma vez) conferir que os tipos de `docChecklist` batem com o enum vivo de `criar_pendencia`)

7. Como validar cada critério de aceite

#	CRITÉRIO	COMO VALIDAR
✓	6.1/2.6 grava, segmenta sem cortar, upload, lincado ao cliente	Ficha → “Áudios/Transcrições” → Gravar (>10 min ou <code>rotateMs</code> reduzido) → blocos “✓ salvo” → tocar; no banco: linhas <code>audio_atendimento</code> agrupadas por <code>/atendimento/<sessão>/</code>
✓	6.2 resumo estruturado gerado e salvo, visível	Após deploy: ficha com sessão de chat → aba Resumo → “Gerar resumo” → campos aparecem; Histórico mostra o evento; dado ausente = “não informado”
✓	6.3 comando gera checklist + pendências, cobrança, bloqueio no protocolo	Após deploy + flag + <code>allowed_tools</code> : no chat “Para Crefisa, solicite extrato e contrato” → cartão de confirmação → confirmar → uma pendência por documento (aba Pendências); entrada não trava

8. Observações finais

- Segurança:** chave do LLM nunca vai ao cliente (resolvida no edge); escritas usam a identidade do usuário (RLS `is_recepcao_or_socio`); `uploaded_by` preenchido em todo insert.
- Isolamento de risco:** 6.2 é uma função nova isolada; 6.3 é gated por flag OFF independente do `CHAT_TOOLS` — deployar não muda nada até ligar.
- Incidente de ambiente (resolvido):** as sessões paralelas (Trilhas A/B/D) compartilham o mesmo diretório e a `main` avançou durante o trabalho; o PR foi reconciliado com a `main` (merge, 1 conflito de flag no `chat-orchestrator` resolvido mantendo ambas) e todo o trabalho ficou isolado num *git worktree* dedicado.

- **Minor deixados como follow-up (não bloqueiam):** testes para `pickAudioMime / isRecordingSupported`; blob órfão se o insert falhar após upload (mitigado por `upsert:true`); `startedAt` de sessão sem `notes` em linhas legadas.

Gerado por Claude Code · TRILHA C · PR #82 (merge `76ab819`) · specs e planos em `docs/superpowers/` · relatório para validação humana.